Алгоритми та їх властивості

- **Алгоритм** – це логічна зв’язана послідовність кроків виконання визначених дій з урахуванням поставлених умов та вхідних даних для досягнення цільового результату.
- **Алгоритм** - це послідовність математичних, логічних або разом узятих операцій, що відрізняються детерменированностью, масовістю, спрямованістю й, що приводить до рішення всіх завдань даного класу за кінцеве число кроків
- **Алгоритм** – послідовність дій, які потрібно виконати для оброблення сукупності початкових даних і здобуття відповідних результатів.
- **Алгоритм** — заздалегідь задане зрозуміле і точне розпорядження можливому виконавцеві зробити певну послідовність дій для отримання рішення задачі за кінцеве число кроків.
- **Алгоритм** — це послідовність дій, направлених на отримання певного результату за кінцеве число кроків.
- **Алгоритм** – набір систематизованих правил виконання обчислювального процесу, що обов'язково приводить до розв'язання певного класу задач після скінченного числа операцій
- **Алгоритм** – це довільна коректно визначена обчислювальна процедура, на **вхід** якої подається деяка величина або набір величин, а результатом виконання якої є **вихідна** величина або набір значень.

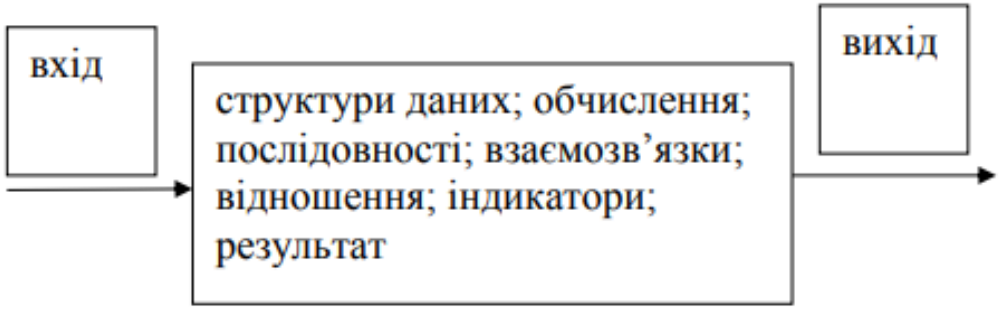
Позначення дій в алгоритмі називається командами або **інструкціями**.

Модель алгоритму

Алгоритми не можуть існувати без структур даних.

Якщо ми знаємо тільки вхід і вихід, модель алгоритму може бути представлена у вигляді чорної скриньки

Якщо ж ми знаємо правила перетворення та обробки вхідних даних, то така скринька стає білою.



Алгоритм. Основні риси (властивості) алгоритму

Алгоритм — це опис порядку дій, які треба виконати, щоб розв’язати деяку задачу. Позначення дій в алгоритмі називаються **командами** або **інструкціями** (statement).

Алгоритми мають кілька загальних властивостей:

- **Зрозумілість** – коли виконавець алгоритму може правильно зрозуміти та виконати команди, записані в алгоритмі;
- **Детермінованість** (однозначність) - кожна команда в алгоритмі має бути чіткою й однозначною;
- **Дискретність** (перервність, роздільність)- алгоритм задає послідовність дій, чітко відокремлених одна від одної;
- **Завершеність** та результативність - обов'язкове здобуття результату після реалізації алгоритму, причому за скінчений час;
- **Масовість** - можливість застосування певного алгоритму до цілого класу однотипних задач. Алгоритм має використовуватися для різних вихідних даних;
- **Виконуваність** (скінченність) – алгоритм завжди можна виконати до кінця. Процес, заданий алгоритмом має тривати скінченний час і не повинен обриватись без отримання результату.

Правила побудови алгоритму

- При побудові алгоритму насамперед необхідно задати об'єкти, з якими буде працювати алгоритм. Алгоритм приступає до роботи з деяким набором даних, які називаються вхідними, і в результаті своєї роботи видає дані, які називаються вихідними
- Для роботи алгоритму потрібна пам'ять. У пам'яті розміщаються вхідні дані, з якими алгоритм починає працювати, проміжні дані й вихідні дані, які є результатом роботи алгоритму.
- Дискретність. Алгоритм будується з окремих кроків (дій, операцій, команд).
- Детермінованість. Після кожного кроку необхідно вказувати, який крок виконується наступний, або давати команду зупинки.
- Збіжність (результативність). Алгоритм повинен завершувати роботу після кінцевого числа кроків. При цьому необхідно вказати, що вважати результатом роботи алгоритму.

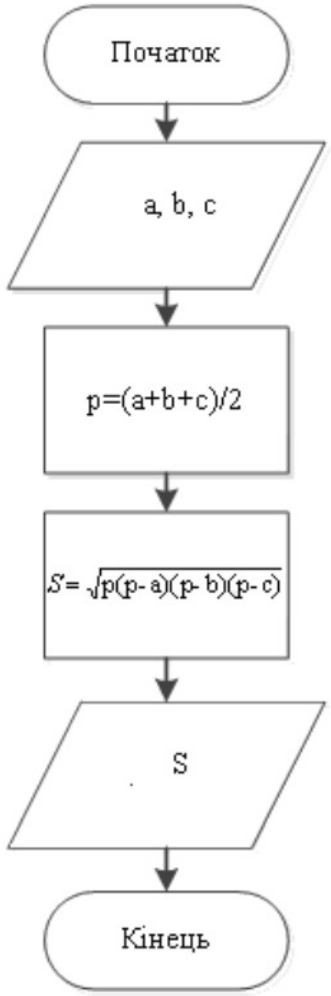
Способи опису алгоритмів

- словесно-формульний;
- графічний у вигляді блок-схеми;
- псевдокодом;
- програмний.

Блок-схема – це графічне зображення, яке демонструє послідовність основних кроків виконання програми.

Блок-схема

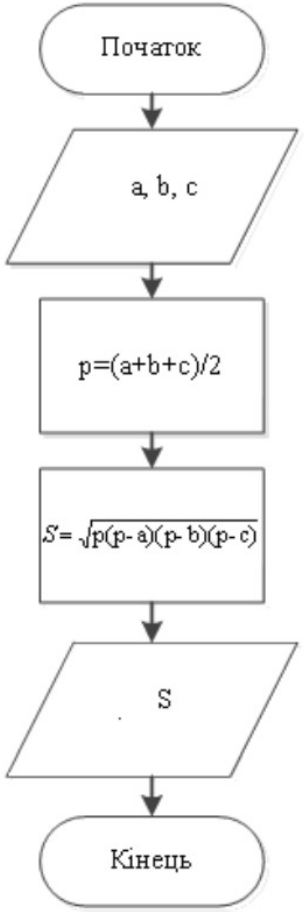
	Блок, що зображає початок чи кінець алгоритму; всередині блока слід написати «початок» або «кінець»
	Блок введення-виведення даних зображується паралелограмом. Текст усередині блока конкретизує операцію, що виконується, і містить слово «введення» або «виведення», а також імена змінних, які необхідно ввести чи вивести
	Арифметичний операторний блок зображується прямокутником. Використовується для позначення дій, що задають або змінюють значення величин. Найчастіше всередині блока записуюсь вираз з використанням математичних символів
так ні	Логічний операторний блок (блок прийняття рішення) зображується ромбом. Використовується для вибору одного з двох можливих напрямків виконання алгоритму (стрілки з позначенням «так» і «ні»). Всередині блока записується умова вибору, перехід по стрілці з позначкою «так» відбувається, коли умова виконується, а перехід по стрілці з позначкою «ні» - у протилежному випадку
	Блоки початку та кінця циклу. Всередині блока початку циклу записують діапазон та крок зміни параметра циклу, а у блоці кінця циклу – вказується ім'я параметра циклу
	Блок виклику піктограми (функції, модуля), тобто допоміжних алгоритмів, які визначені автономно. Всередині блока записується, наприклад, ім'я функції, яка викликається, та список фактичних параметрів.
	Символ, що використовується для зв'язку елемента схеми з коментарем
	Символ-з'єднувач застосовується для обриву лінії схеми та продовження її у другому місці



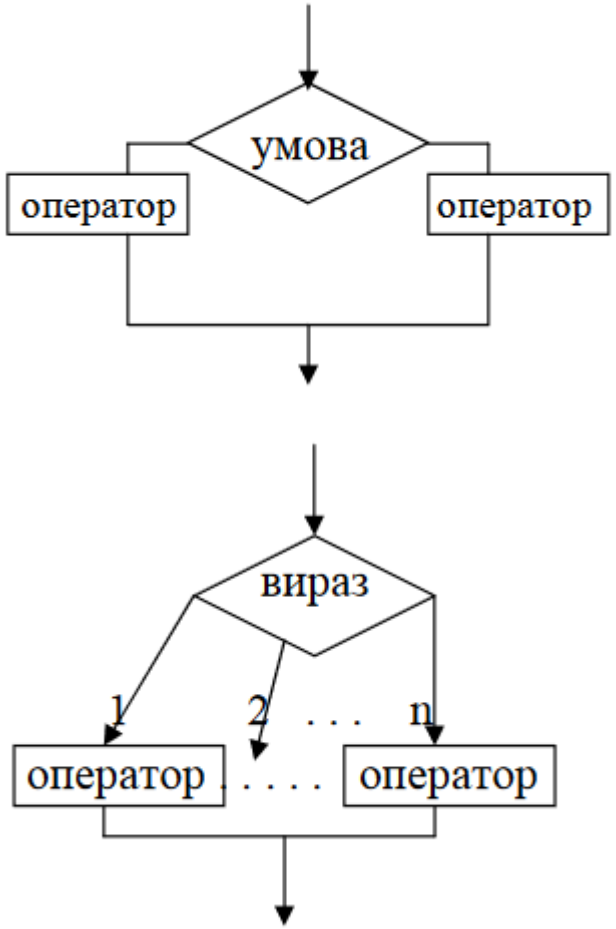
Типові структури алгоритму

- **лінійна** (операції виконуються послідовно, по черзі їх запису);
- **розгалужена** (існує умова, залежно від виконання якої є кілька напрямків обчислень. Якщо напрямків два, то це простий розгалужений алгоритм, якщо більше – складний);
- **циклічна** (процес з одним або більше блоками, що повторюються).

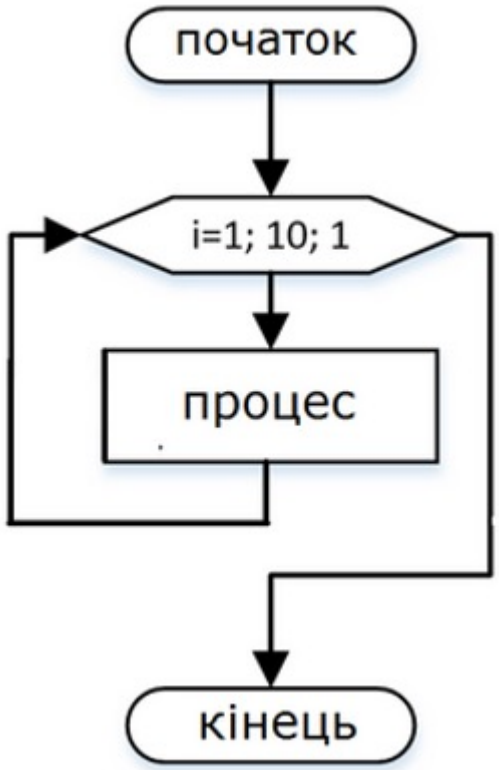
Лінійна форма



Розгалужена форма

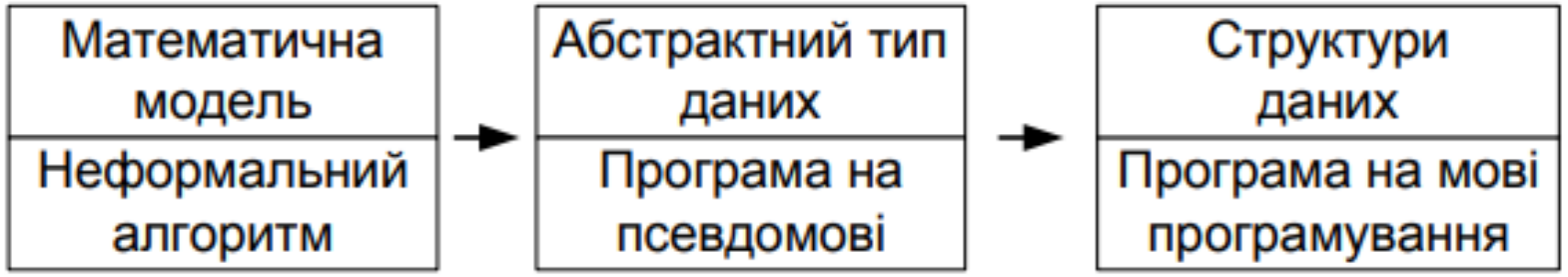


Циклічна форма



Покрокове проектування алгоритмів

Схематично узагальнений процес програмування можна представити наступною схемою:



Покрокове проектування алгоритмів

На **першому** етапі створюється модель вихідної задачі, для чого застосовуються відповідні математичні моделі. На цьому етапі для знаходження рішення також будується неформальний алгоритм.

На **наступному** етапі алгоритм записується на псевдомові – композиції конструкцій мови програмування і менш формальних і узагальнених операторів на простій мові. Продовженням цього етапу є заміна неформальних операторів послідовністю більш детальних і формальних операторів. З цієї точки зору програма на псевдомові повинна бути достатньо детальною, так як в ній фіксуються (визначаються) різні типи даних, над якими виконуються оператори. Потім створюються абстрактні типи даних для кожного зафіксованого типу даних (за винятком елементарних типів даних, таких як цілі й дійсні числа або символьні стрічки) шляхом завдання імен функцій для кожного оператора, який працює з даними абстрактного типа, і заміни їх (операторів) викликом відповідних функцій.

Третій етап процесу – програмування – забезпечує реалізацію кожного абстрактного типа даних і створення функцій для виконання різних операторів над даними цих типів. На цьому етапі також замінюються всі неформальні оператори псевдомови на код мови програмування. Результатом цього етапу повинна бути виконувана програма. Після її наладки отримують працюючу програму.

Аналіз алгоритму

- **Поведінка алгоритму.** Чи знаходить він найкраще з можливих або просто гарне рішення? Чи може бути кілька найкращих рішень? Чи є сенс віддати перевагу одному з них?
- **Швидкість алгоритму.** Швидкий він чи ні? Чи уповільнює роботу тільки з деякими вхідними даними?
- **Вимоги до пам'яті алгоритму**. Скільки комп'ютерних ресурсів йому необхідно? Чи є такий обсяг прийнятним? Чи потрібні алгоритму мільйони терабайт пам'яті, якими комп'ютер не володіє (принаймні, зараз)?
- **Основні методи, використовувані в алгоритмі**. Чи можна задіяти їх повторно для вирішення подібних завдань?

Аналіз алгоритму

Після того, як алгоритм реалізований, проводять його аналіз. У результаті цього можна отримати відповідь на питання чи підходить алгоритм для розв’язання поставленої задачі. Аналіз полягає у визначенні та аналізі таких критеріїв як:

- час роботи програми, як функцію від вхідних даних;
- загальну кількість пам’яті, що необхідна для даних програми;
- загальний об’єм програмного коду;
- чи програма розв’язує коректно (правильно) поставлену задачу;
- чи стресо-стійка програма, тобто як добре буде поводити себе програма з некоректними вхідними даними.
- комплексність програми, тобто чи легко програму читати, розуміти та модифікувати.

Останні чотири пункти частіше за все покладаються на зовнішні аналізатори (інженери-тестувальники, автоматичні системи тестування, рецензування коду), а перші два цілковито знаходяться у зоні відповідальності програміста, який створює програму.

Час виконання алгоритму

Як оцінити на скільки оптимальним (швидким) є цей алгоритм:

- Як оцінити швидкодію цього алгоритму?
- У яких одиницях цю швидкодію вимірювати?
- Яка врешті буде складність цього алгоритму?

Швидкодію алгоритму можна оцінити як «час виконання програми» побудованої за цим алгоритмом, але як порахувати цей час?

Для оцінки складності алгоритму необхідно використовувати критерій, що не залежить від потужності ЕОМ або мови програмування на якій реалізовано алгоритм.

Робота будь-якої програми, що виконується комп'ютером складається з елементарних операцій - присвоєння, елементарні арифметичні операції, операціїпорівняння, виклик функції/методу, звернення за індексом до елементу списку (масиву) тощо.

Часом виконання програми (running time) будемо називати кількість елементарних операцій, які виконує комп'ютер під час виконання програми.

Час виконання алгоритму

Час виконання програми будемо позначати

$$T(n)$$

де n – розмір вхідних даних.

$T(n)$ представляє собою функцію часу від вхідних даних, її ще називають **часовою складністю алгоритму**.

Часом виконання програми у найгіршому випадку (worst-case running time) будемо називати найбільшу кількість елементарних операцій, які виконує комп'ютер під час виконання програми для довільних вхідних даних розміру n.

Часом виконання програми у найкращому випадку (best-case running time) будемо називати найменшу кількість елементарних операцій, які виконує комп'ютер під час виконання програми яка досягається для деякого набору вхідних даних розміру n.

Часом виконання програми в середньому (eng. average-case running time, average-case time complexity) будемо називати усереднену кількість елементарних операцій, які виконує комп'ютер під час виконання програми для всіх наборів вхідних даних розміру n.

Час виконання алгоритму на прикладі

Приклад. Визначити чи заданий елемент міститься у списку, що містить n елементів.

```
1   static bool FindElement(int[] myArray, int myElement, int n){
2   int i = 0;
3   while (i < n) {
4       if (myElement == myArray[i]) {
5           return true; }
6       i++; }
7       return false;
8   }
```

Рядок	Час	
	у найкращому випадку	у найгіршому випадку
2	2	2
3	3	$3(n+1)$
4	5	$5n$
5	2	–
6	–	$4n$
7	–	2
$T(n)$	12	$12n+7$

На практиці час виконання у середньому знайти значно складніше, аніж час виконання у найгіршому випадку. Тому в основному використовують час виконання у найгіршому випадку, як характеристику часової складності алгоритму.

Складність алгоритму

Складність алгоритму – кількісна характеристика, що відображає споживані алгоритмом ресурси під час свого виконання.

Часова складність алгоритму – показник ефективності алгоритму з точки зору часу його виконання.

Складність алгоритму **по пам'яті** – показник ефективності алгоритму з точки зору пам'яті, яку використовує алгоритм.

Часова складність алгоритму

Слід зауважити, що, як правило, нікого не цікавить швидкодія алгоритму при невеликих значеннях вхідних даних – для вхідних даних малого розміру час виконання будь-якого алгоритму є задовільним з практичної точки зору. Тому при оцінці швидкодії алгоритмів найбільше цікавить поведінка T(n) при великому розміру вхідних даних (великих значеннях n).

Під час оцінки швидкодії алгоритму при великих значеннях розміру вхідних даних, як правило цікавить не точна кількість операцій, яку здійснює програма, а порядок цієї кількості операцій.

Тому при оцінці швидкодії програми:

- У формулі кількості операцій $T(n)$, враховують лише доданок, що зростає найшвидше.
- Сталий множник при цьому доданку встановлюють рівними 1.

Отриману величину називають **порядком складності алгоритму** або **асимптотичною складністю**.

Тобто для алгоритму з часом виконання $T(n) = 345n^3 + 123n^2 + 98n + 15$ порядок складності буде n^3

Фактично тут іде мова про **асимптотичну поведінку** функцій часу виконання, що залежать від вхідних даних алгоритму. Це означає, що нас цікавить тільки те, як час роботи алгоритму зростає зі збільшенням розміру вхідних даних гранично, тобто коли цей розмір зростає до нескінченності. Іншими словами, асимптотична складність являє собою ліміт T(n) при $n \to \infty$

Складність алгоритму по пам’яті

Аналіз складності по пам’яті відбувається так само, як і аналіз часової складності, тільки замість кількості операцій аналізується пам’ять, яку використовує програма.

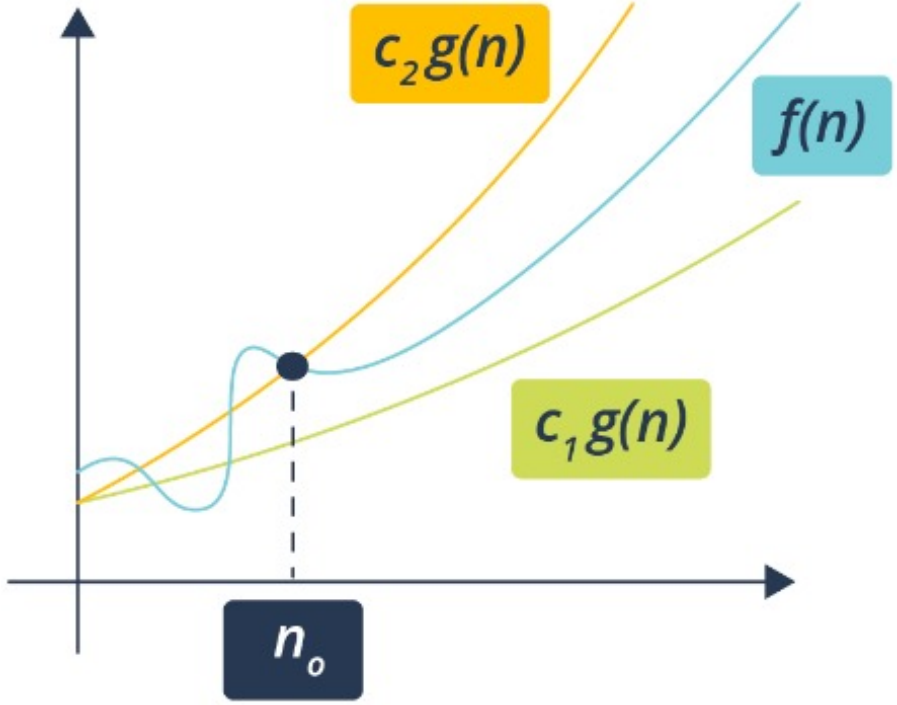
Пам’ять вимірюється з точки зору найбільшого використання пам’яті програмою після її запуску. Тобто оцінюється максимальна величина пам’яті, яка виділялась під час всього процесу виконання алгоритму.

Θ-нотація

Для деякої функції $g(n)$ запис $\Theta(g(n))$ (читається «тета від n^2») означає множину функцій:

$$\Theta(g(n)) = \left\{ \begin{array}{c} f(n): \text{існують додатні константи } c_1, c_2 \text{ та } n_0, \text{такі що} \\ 0 \le c_1 g(n) \le f(n) \le c_2 g(n) \text{ для всіх } n \ge n_0 \end{array} \right\}$$

Функція $f(n)$ належить множині $\Theta(g(n))$, якщо існують додатні константи c_1 та c_2, які дозволяють заключити цю функцію в межі між функціями $c_1 g(n)$ та $c_2 g(n)$ для достатньо великих n. Оскільки $\Theta(g(n))$ – це множина, то можна записати $f(n) \in \Theta(g(n))$, але також більш поширеним є запис $f(n) = \Theta(g(n))$.



На рисунку показане інтуїтивне зображення функцій $f(n)$ та $g(n)$, таких що $f(n) = \Theta(g(n))$. Кажуть, що функція $g(n)$ є асимптотично точною оцінкою функції $f(n)$.

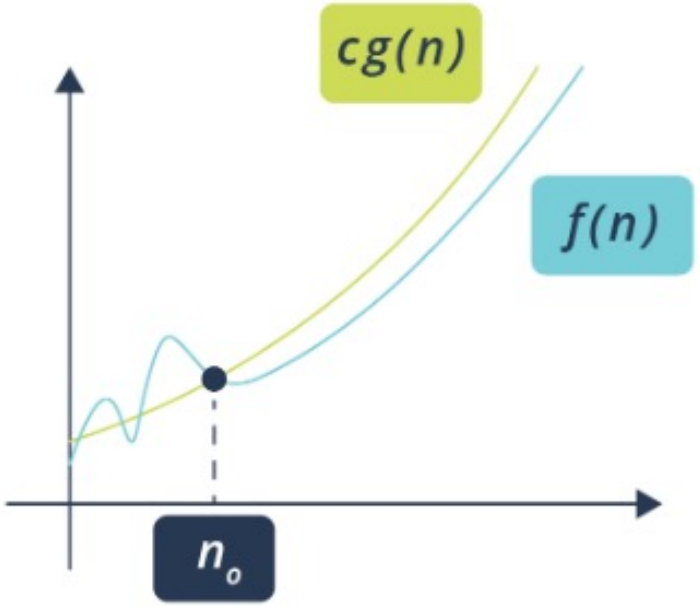
О-нотація

У Θ-позначеннях функція асимптотично обмежується знизу та згори. Якщо ж достатньо визначити тільки асимптотичну нижню границю, то використовують О-позначення (читається як «омікрон» або як «велике о»).

О-нотація визначає функцію f(n), яка показує, як буде змінюватись складність алгоритму при зміні кількості вхідних даних у найгіршому для алгоритму випадку.

Визначення. Говорять, що функція часу T(n) має порядок росту O(f(n)), якщо існують константи c і n_0 такі, що для усіх $n \geq n_0$ виконується нерівність $T(n) \leq cf(n)$

$$O(g(n)) = \{f(n) : \text{існують додатні константи } c \text{ та } n_0, \text{такі що } 0 \leq f(n) \leq cg(n) \text{ для всіх } n \geq n_0\}$$



О-нотація

Легко помітити, що якщо $f(n) = \Theta(g(n))$, то $f(n) = O(g(n))$, оскільки Θ-позначення більш сильне, ніж О-позначення (в термінах теорії множин це можна записати як $\Theta(g(n)) \subset O(g(n))$).

Таким чином, доведення того, що функція $an^2 + bn + c$, де $a>0$, належить множині $\Theta(n^2)$, одночасно доводить, що множині $O(n2)$ належить будь-яка подібна квадратична функція. Але так само будь-яка лінійна функція $an + b$ при $a>0$ також належить множині $O(n^2)$, що легко показати, обравши $c = a + |b|$ та $n0 = \max(1, –b/a)$.

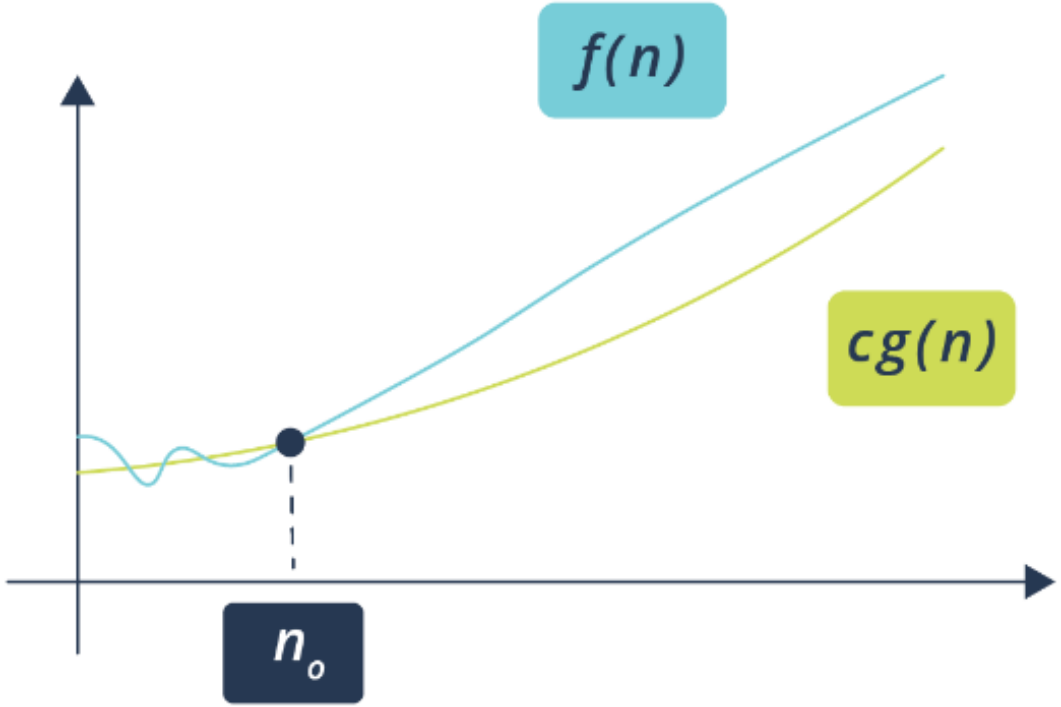
Оскільки О-позначення описують верхню межу, то під час їх використання для обмеження часу роботи алгоритму в найгіршому випадку отримується верхня межа цієї величини для будь-яких вхідних даних.

Таким чином, межа $O(n^2)$ для часу роботи алгоритму в найгіршому випадку може застосовуватись для часу розв’язку задачі з будь- якими вхідними даними, чого не можна сказати про Θ-позначення. Наприклад, оцінка $\Theta(n^2)$ для часу сортування включенням в найгіршому випадку не може застосовуватись для довільних даних: якщо вхідні дані вже відсортовані, то час роботи цього алгоритму оцінюється як $\Theta(n)$.

Ω-нотація

Аналогічно тому, як в O-позначеннях дається асимптотична верхня границя функції, в Ω-позначеннях дається її асимптотична нижня границя. Для заданої функції $g(n)$ позначення $\Omega(g(n))$ (читається як «омега від $g(n)$») позначає множину функцій, таких що:

$$\Omega(g(n)) = \{f(n) : \text{існують додатні константи } c \text{ та } n_0, \text{такі що } 0 \le cg(n) \le f(n) \text{ для всіх } n \ge n_0\}$$



Використовуючи введені означення асимптотичних позначень, легко довести, що для довільних двох функцій $f(n)$ та $g(n)$ відношення $f(n) = \Theta(g(n))$ виконується тоді й тільки тоді, коли $f(n) = O(g(n))$ та $f(n) = \Omega(g(n))$

Оскільки Ω-позначення використовуються для визначення нижньої границі часу роботи алгоритму в найкращому випадку, то вони також дають нижню границю часу роботи алгоритму для довільних вхідних даних.

Нотації асимптотичної складності

Позначення	Межі	Характер
Θ (Тета)	Нижня та верхня границі, точна оцінка	Дорівнює
O (О-велике)	Верхня границя, точна оцінка невідома	Менше або дорівнює
o (о-мале)	Верхня границя, не точна оцінка	Менше
Ω (Омега-велике)	Нижня границя, точна оцінка невідома	Більше або дорівнює
ω (Омега-мале)	Нижня границя, не точна оцінка	Більше

Складність	Ефективність
Θ(N)	= N
O(N)	<= N
o(N)	< N
ОМЕГА(N)	>= N
ω(N)	> N

Якщо алгоритм має складність ___, тоді його ефективність ___

Використання О-нотації

Властивості, наведені вище дозволяють сформулювати прості правила для визначення асимптотичної поведінки функції.

- Мультиплікативні константи можна опускати, тобто для будь-якої додатної сталої C

$$Cn^3 = O(n^3);$$

- Нехай $b \geq a$ тоді

$$n^a = O(n^b);$$

- Нехай $a > 1$. Тоді для будь-якого $k \geq 1$,

$$n^k = O(a^n);$$

- Для будь-якого $k \geq 1$

$$\log^k n = O(n);$$

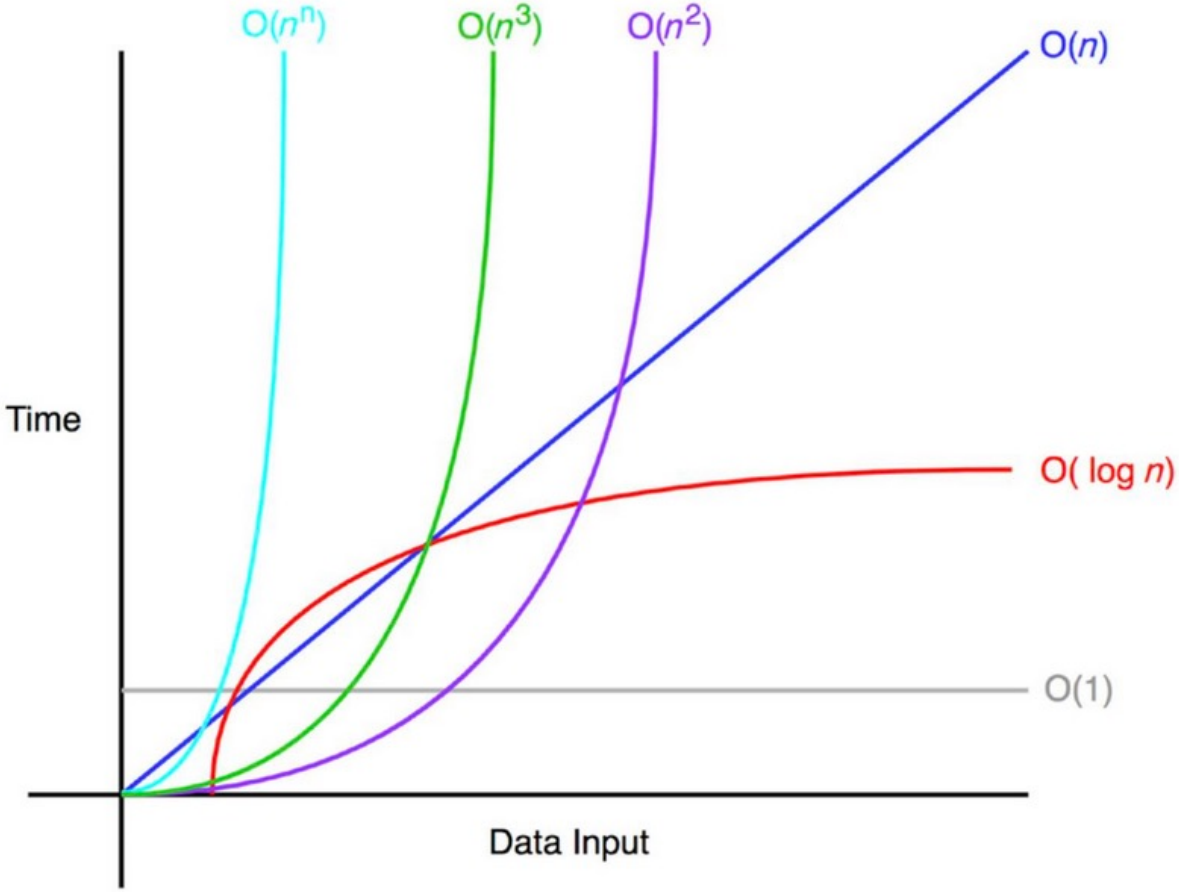
- Якщо функція є сумою кількох функцій, то її асимптотична поведінка визначається доданком, що зростає найшвидше. Наприклад

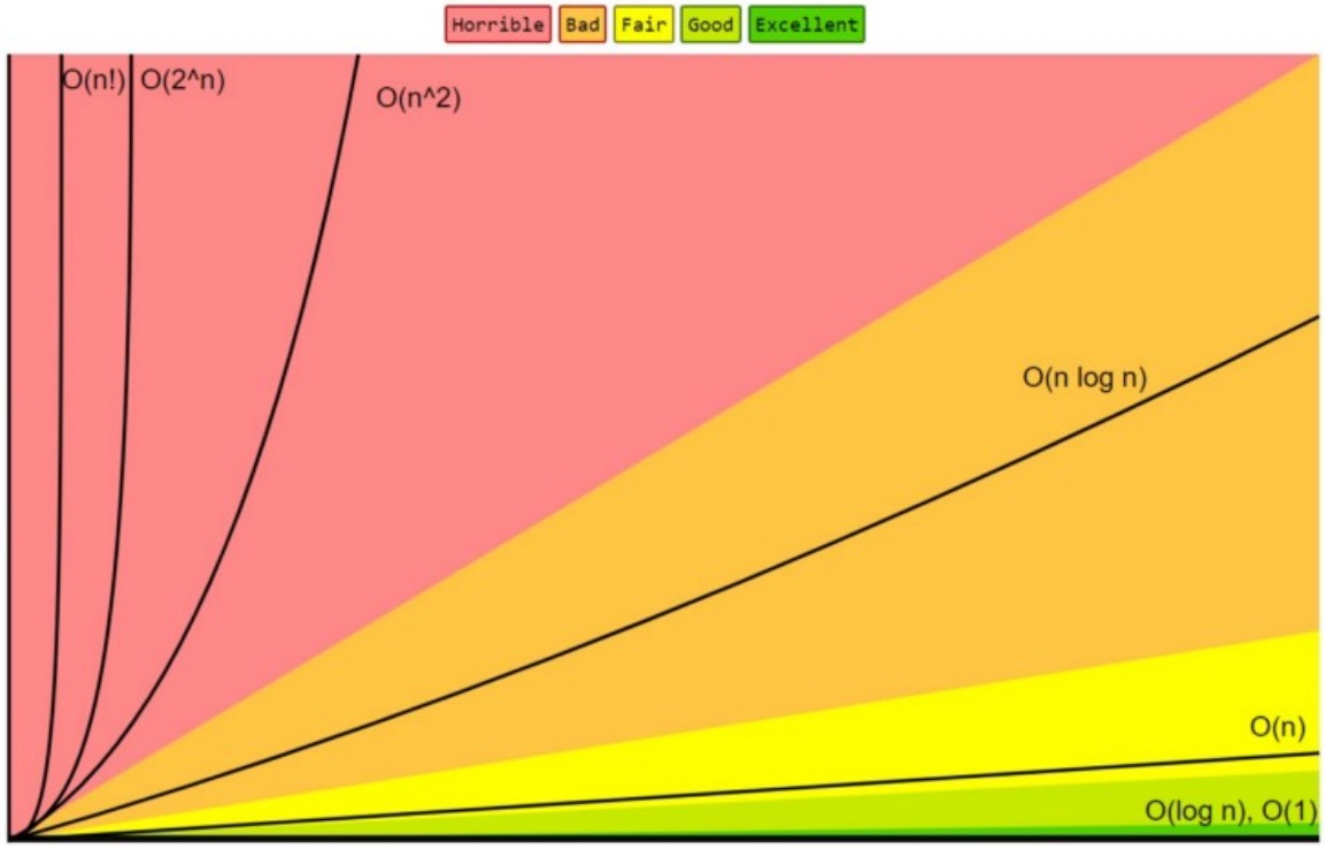
$$345n^3 + 123n^2 + 98n + 15 = O(n^3).$$

Поширені асимптотичні складності

Складність	Коментар	Приклади
$O(1)$	Сталий час роботи не залежно від розміру задачі	Пошук у хеш-таблиці
$O(\log\log n)$	Дуже повільне зростання необхідного часу	Очікуваний час роботи інтерполюючого пошуку n елементів
$O(\log n)$	Логарифмічне зростання — подвоєння розміру задачі збільшує час роботи на сталу величину	Швидке обчислення x^n; двійковий пошук у відсортованому масиві з n елементів
$O(n)$	Лінійне зростання — подвоєння розміру задачі подвоїть і необхідний час	Додавання/віднімання чисел з n цифр; лінійний пошук в масиві з n елементів
$O(n\log n)$	Лінеаритмічне зростання — подвоєння розміру задачі збільшить необхідний час трохи більше ніж вдвічі	Сортування злиттям або купою масиву з n елементів.
$O(n^2)$	Квадратичне зростання — подвоєння розміру задачі вчетверо збільшує необхідний час	Елементарні алгоритми сортування масивів з n елементів; Лінійний пошук у квадратній матриці розмірності n.
$O(n^3)$	Кубічне зростання — подвоєння розміру задачі збільшує необхідний час у вісім разів	Звичайне множення матриць
$O(a^n)$	Експоненціальне зростання — збільшення розміру задачі на 1 призводить до a-кратного збільшення необхідного часу; подвоєння	Деякі задачі комівояжера; Алгоритми пошуку повним перебором

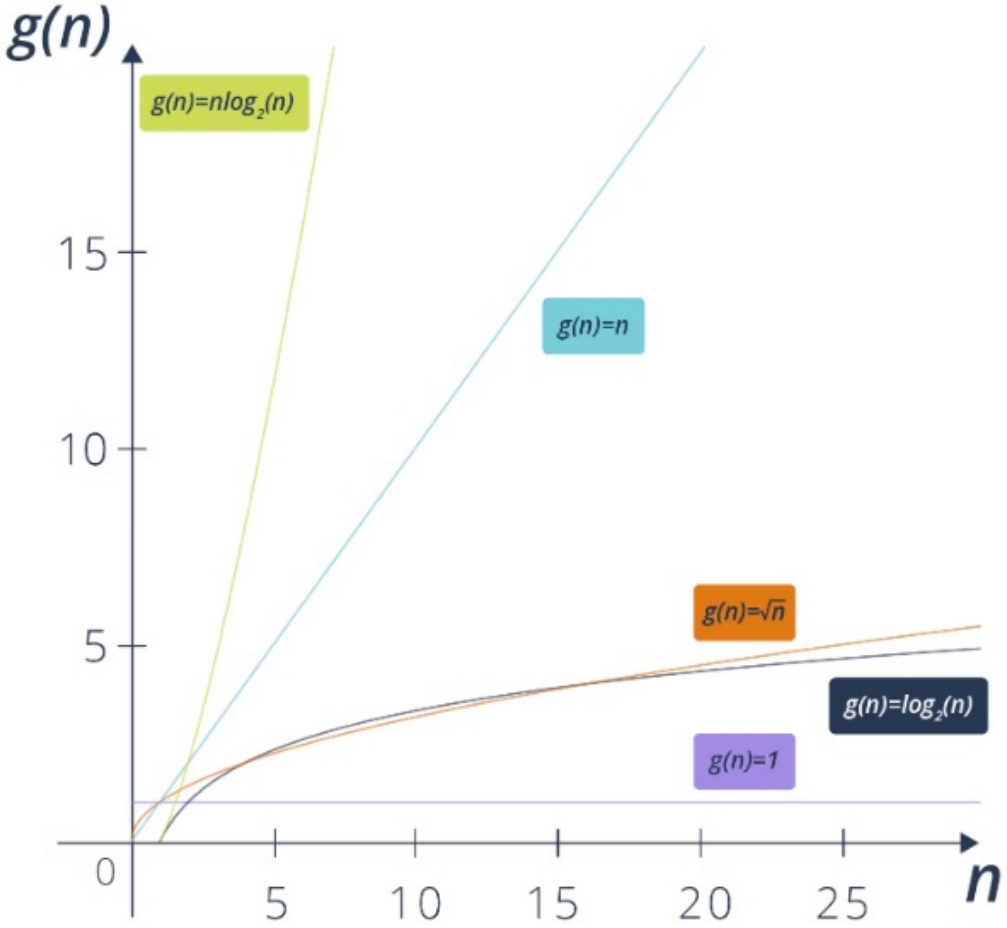
Графік О-нотації

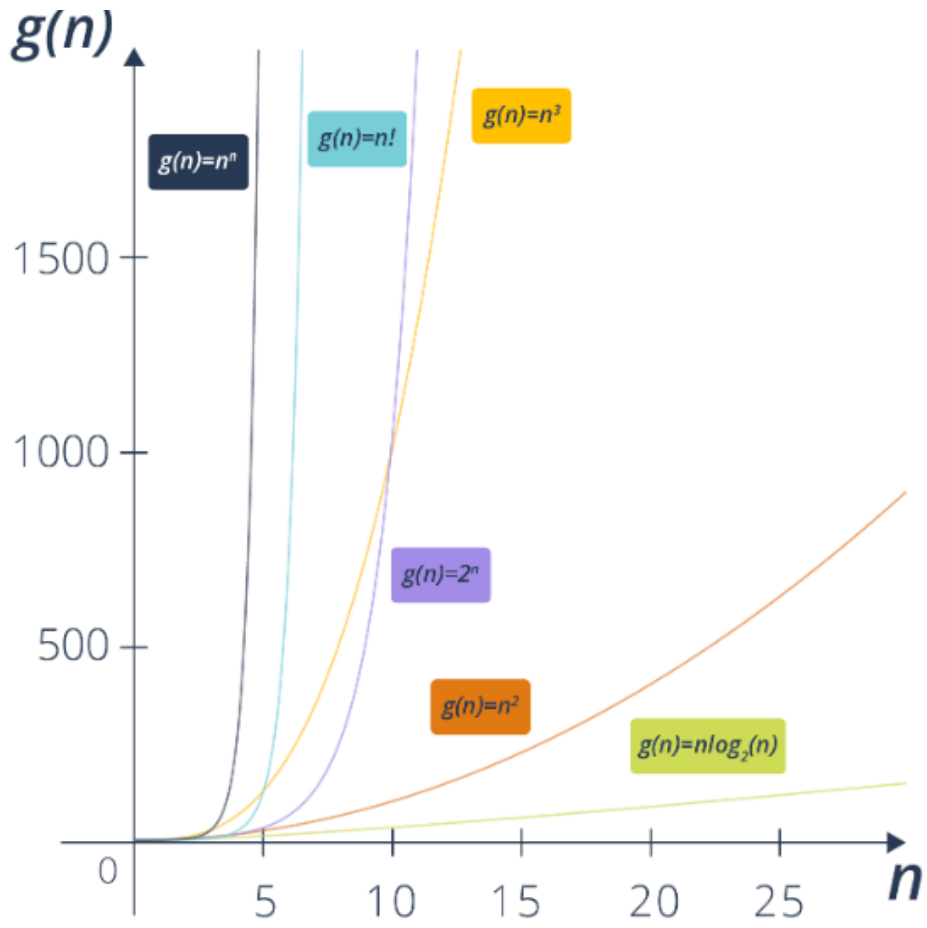




Графік О-нотації

$$1 < \log_2(n) < \sqrt{n} < n < n\log_2(n) < n^2 < n^3 < \cdots < 2^n < n! < n^n$$





$$1 < \log_2(n) < \sqrt{n} < n < n\log_2(n) < \cdots$$

$$\cdots < n\log_2(n) < n^2 < n^3 < \cdots < 2^n < n! < n^n$$

Правила оцінки алгоритмів

Найгірший час виконання алгоритму, що складається з послідовності виразів

S1
S2
...
Sm

має асимптотичну складність

$$O(\max\{T1(n),\ldots,Tm(n)\}),$$

де $T1(n),\ldots,Tm(n)$ – час виконання відповідно інструкцій `S1`, ..., `Sm`.

Правила оцінки алгоритмів

Найгірший час виконання алгоритму, що містить цикл **while**

має асимптотичну складність

$$O(\max\{T1(n)\times(I(n)+1), T2(n)\times I(n)\}),$$

де $T1(n)$,$T2(n)$ – час виконання відповідно інструкцій `S1`, `S2`, а $I(n)$ – кількість ітерацій циклу, що виконується у найгіршому випадку.

Правила оцінки алгоритмів

Найгірший час виконання алгоритму, що містить цикл **for**

має асимптотичну складність

$$O(\max\{T1(n)\times(I(n)+1), T2(n)\times I(n)\}),$$

де $T1(n)$,$T2(n)$ – час виконання відповідно інструкцій `S1`, `S2`, а $I(n)$ – кількість ітерацій циклу, що виконується у найгіршому випадку.

Правила оцінки алгоритмів

Найгірший час виконання алгоритму, що містить **умовний оператор**

має асимптотичну складність

$$O(\max\{T1(n), T2(n), T3(n)\}),$$

де $T1(n), T2(n), T3(n)$ – час виконання відповідно інструкцій `S1`, `S2`, `S3`.

Асимптотична складність для вхідних даних різного розміру

Складність	$n = 1$	$n = 8$	$n = 1K$	$n = 1024K$
$O(1)$	1 нс	1 нс	1 нс	1 нс
$O(\log n)$	1 нс	3 нс	10 нс	20 нс
$O(n)$	1 нс	8 нс	102 нс	1.05 мс
$O(n \log n)$	1 нс	24 нс	1.02 мс	21 мс
$O(n^2)$	1 нс	64 нс	10.2 мс	18.3 хв
$O(n^3)$	1 нс	512 нс	1.07 с	36.5 р.
$O(2^n)$	1 нс	256 нс	10^{292} р.	10^{10^5} р.

Користь швидких алгоритмів

Складність алгоритму і визначає те збільшення розміру задачі, яке можна досягти зі збільшенням швидкості комп'ютера.

Є п'ять алгоритмів A1 – A5 з деякими часовими складностями.

Нехай одиницею часу буде одна мілісекунда. Тоді алгоритм *A1* може обробити за одну секунду вхід розміром 1000, в той час як *A5* – вхід розміром не більше 9.

Алгоритм	Часова складність	Максимальний розмір входу задачі		
		1 секунда	1 хвилина	1 година
A_1	N	1000	$6 \cdot 10^4$	$3{,}6 \cdot 10^6$
A_2	$n \log n$	140	4893	$2 \cdot 10^5$
A_3	n^2	31	244	1897
A_4	n^3	10	39	153
A_5	2^n	9	15	21

Користь швидких алгоритмів

Припустимо, що наступне покоління комп'ютерів буде у 10 разів швидше нинішнього. У таблиці показано як зростуть розміри задач, які можна розв'язати завдяки такому збільшенню швидкості.

Алгоритм	Часова складність	Максимальний розмір задачі	
		до прискорення	після прискорення
A_1	n	S_1	$10\ S_1$
A_2	$n \log n$	S_2	$\approx 10\ S_2$ *для більших* S_2
A_3	n^2	S_3	$3{,}16\ S_3$
A_4	n^3	S_4	$2{,}15\ S_4$
A_5	2^n	S_5	$S_5+3{,}3$

Якщо за основу для порівняння взяти 1 хвилину, то, замінюючи алгоритм *A4* алгоритмом *A3*, можна розв'язати задачу, в 6 разів більшу, а замінюючи *A4* на *A2* – більшу в 125 разів. Ці результати набагато кращі, ніж дворазове поліпшення, що досягається за рахунок десятикратного збільшення швидкості.

Приклади коду

O(1) – час виконання є константою:

```
public int GetCount(int[] items)
{
    return items.Length;
}
```

O(n) – час виконання міняється лінійно:

```
public long GetSum(int[] items)
{
    long sum = 0;
    foreach (int i in items)
    {
        sum += i;
    }
 return sum;
}
```

Визначення часової складності на практиці

Завдання 1.

```
int n = Convert.ToInt32(Console.ReadLine());
int a = n / 100;
int b = n / 10 % 10;
int c = n % 10;
int revert = c * 100 + b * 10 + a;
Console.WriteLine(revert);
```

Розв’язок.

При збільшенні вхідного параметра n кількість операцій не змінюється, тобто в усіх випадках кількість операцій є постійною величиною, тому асимптотична складність O(1).

Визначення часової складності на практиці

Завдання 2.

```
int n = Convert.ToInt32(Console.ReadLine());
int count = 0;
for (int i = 0; i < n; i++)
{
        int number = Convert.ToInt32(Console.ReadLine());

        if (number % 10 == 0)
        {
                count = count + 1;
        }
}
Console.WriteLine(count);
```

Розв’язок.

Кількість операцій залежить від n. Якщо збільшити n в 10 разів, то кількість операцій збільшиться не більш ніж в 10 разів. Отже, асимптотична складність лінійна, тобто O(n).

Визначення часової складності на практиці

Завдання 3.

```
int n = Convert.ToInt32(Console.ReadLine());
int count = 0;
int i = 1;
while (i <= n * n)
{
        count++;
        i = i * 3;
}
Console.WriteLine(count);
```

Розв’язок.

Складність даного алгоритму буде $O(\log_3 n^2)$.
Властивості алгоритмів:
1. $\log_a n = c * \log_b n$
2. $\log n^2 = 2 * \log n$
Тобто, $O(\log_3 n^2)$ *приводимо до* $O(\log n)$.

Визначення складності по пам’яті на практиці

Завдання 4.

```
int n = Convert.ToInt32(Console.ReadLine());
int[] array = new int[n];
for (int i = 0; i < n; i++)
{
        array[i] = i * 2;
}
```

Розв’язок.

Цикл має n ітерацій. Тобто часова складність рівна O(n).
Даний фрагмент коду створює масив розміром n, цикл має n ітерацій. Тому складність по пам’яті рівна O(n).

Визначення складності по пам’яті на практиці

Завдання 5.

```
int n = Convert.ToInt32(Console.ReadLine());
int sum = 0;
for (int i = n * n; i > 1; i--)
{
        n = Convert.ToInt32(Console.ReadLine());
        sum += n;
}
Console.WriteLine(sum);
```

Розв’язок.

Цикл має n^2 ітерацій. Тобто часова складність рівна $O(n^2)$.
Але пам’ять витрачається тільки на три змінні, тому складність по пам’яті рівна $O(1)$.